PUC Minas
MAIOR UNIVERSIDADE CATÓLICA DO MUNDO - Fonte: Vaticano
MELHOR UNIVERSIDADE PRIVADA DO BRASIL - Guia do Estudante, por 6x
ENTRE AS MELHORES UNIVERSIDADES DO MUNDO – Times (Ranking Times High Education)
ÁREA DA COMPUTAÇÃO PUC MINAS: 3º LUGAR PREF.MERCADO - Folha de São Paulo/RUF, 2025

Pontifícia Universidade Católica de Minas Gerais
ICEI – Instituto de Ciências Exatas e Informática
DCC – Departamento de Ciência da Computação
Curso: Ciência da Computação
***Campus:* Coração Eucarístico**
Disc.: Algoritmos e Estruturas de Dados I
Professor: Lúcio Mauro Pereira
Lista de Exercícios nº 6

Introdução aos algoritmos recursivos – Uma abordagem prática

Estudar:

Obra: C: como programar. 8ed. Autor: Deitel.
Disponível na biblioteca da PUC Minas de forma física e *e-book*.
Estudar o Capítulo 5, seções 5.13, 5.14, 5.15: **Recursão**

Para cada problema proposto neste caderno de exercícios:

- *Elaborar um modelo de solução. Expressá-lo através de fluxograma e/ou texto estruturado – algoritmo.*
- *Codificar a solução através da linguagem C.*

1. Construa uma função que escreva na tela a sequência abaixo:
1, 2, 3, 4, 5,
a) Versão iterativa
b) Versão recursiva

2. Construa uma função que escreva na tela a sequência abaixo:
1, 2, 3, 4, 5, 5, 4, 3, 2, 1,
a) Versão iterativa
b) Versão recursiva

3. Construa versões da função anterior de forma que os limites inferior e superior sejam parametrizados.
Teste a sua função a partir da *main()*. Para testá-la, faça de forma que os limites inferior e superior sejam valores lidos. Para isto, construa uma função que leia e retorne um valor inteiro fornecido pelo teclado.
a) Versão iterativa
b) Versão recursiva

4. Construa uma função que escreva em ordem decrescente os inteiros de N até 0.
Argumento da função: um inteiro relativo ao valor de N.
Retorno: vazio
Teste a sua função a partir da função principal. Planeje uma função específica para ler do teclado o valor de N.
a) Versão iterativa
b) Versão recursiva

5. Crie uma função que calcule a soma dos números inteiros de *n* até *m,* sendo *n* e *m* limites inferior e superior, respectivamente.
Planeje, cuidadosamente, sua solução, de forma que as funções implementem funcionalidades atômicas.
a) Versão iterativa
b) Versão recursiva

6. Construa uma função para o algoritmo que trata o problema do cálculo da potenciação.
   a) Versão iterativa
   b) Versão recursiva

7. Construa uma função que calcule o fatorial de um número natural N.
   a) Versão iterativa
   b) Versão recursiva

8. Construa uma função que verifique ser um número natural um palíndromo (capicua) ou não.
   a) Versão iterativa
   b) Versão recursiva

9. Construa algoritmos para calcular o *k-ésimo* termo Fibonacci utilizando as abordagens abaixo. Antes de iniciar a codificação, busque e planeje, cuidadosamente, as estratégias que o algoritmo deverá adotar.
   a) Algoritmo iterativo
   b) Algoritmo recursivo

10. Refaça, no modo iterativo e recursivo, os seguintes problemas:
   a) Escrever um número de forma invertida
   b) Soma dos dígitos de um número natural
   c) Verificar se um número natural é potência de 2 ou não

11. Identificar se um número natural é primo ou não.
   a) Abordagem iterativa
   b) Abordagem recursiva

12. Da obra,

C: como programar, Deitel, resolva o exercício 5.39, **Torres de Hanói**.
Obs.: Pode ocorrer de o número do exercício mudar de acordo com a edição.

(Torres de Hanói) Todo cientista computacional principiante deve lidar com determinados problemas clássicos, e o problema das Torres de Hanói (veja a Fig. 5.18) é um dos mais famosos. Diz a lenda que em um templo do Extremo Oriente os sacerdotes estão tentando mover uma pilha de discos de um pino para outro. A pilha inicial tinha 64 discos colocados em um pino e organizados na ordem decrescente, da base para o topo. Os sacerdotes estão tentando mover a pilha desse pino para um segundo pino com a restrição de que exatamente um disco deve ser movido de cada vez e em nenhum momento um disco maior pode ser colocado sobre um disco menor.

Há um terceiro pino disponível para colocação temporária de discos. Teoricamente o mundo terminará quando os sacerdotes completarem sua tarefa, portanto há pouco estímulo para facilitarmos seus esforços.

Vamos assumir que os sacerdotes estão tentando mover os discos do pino 1 para o pino 3. Desejamos desenvolver um algoritmo que imprimirá a sequência exata de transferências dc discos de um pino para outro. Se fossemos solucionar este problema com os métodos convencionais, rapidamente nos encontraríamos perdidos lidando com os discos. Em vez disso, se solucionarmos o problema com a recursão em mente, ele se torna imediatamente viável. Mover *n* discos pode ser considerado em termos de mover apenas *n - I dis*cos (daí a recursão) como se segue:

1 Mova *n* - 1 discos do pino I para o pino 2. usando o pino 3 como área de armazenamento temporário.
2. Mova o último disco (o maior) do pino 1 para o pino 3.
3. Mova os *n* - 1 discos do pino 2 para o pino 3. usando o pino 1 como área de armazenamento temporário. O processo termina quando a última tarefa envolver mover *n* = 1 disco, i.e., o caso básico. Isso é realizado movendo simplesmente o disco para o seu destino final, sem a necessidade de uma área

de armazenamento temporário.

Escreva um programa para resolver o problema das Torres de Hanói. Use uma função recursiva com quatro parâmetros:
1. O número de discos a serem movidos
2. O pino no qual esses discos estão colocados inicialmente
3. O pino para o qual essa pilha de discos deve ser movida
4. O pino a ser usado como área de armazenamento temporário Seu programa deve imprimir instruções precisas necessárias para mover os discos do pino inicial para o pino de destino. Por exemplo, para mover uma pilha de três discos do pino 1 para o pino 3, seu programa deve imprimir a seguinte sequência de movimentos:

1 —> 3 (Isso significa mover o disco do pino 1 para o pino 3.)
1 => 2
3=>2
1 => 3
2 => 1
2=>3
1 -> 3